Playwright class notes

To Run the test case

Npx playwright test test_name —> headless mode

What is automation testing ?
Automation Testing is a software testing technique where test cases are executed automatically using tools and scripts to validate the functionality of an application without manual intervention.

What is playwright ?
Playwright is an open-source end-to-end automation testing framework developed by Microsoft that is used to automate web applications across multiple browsers.

What are the advantages of playwright ?
Playwright provides cross-browser support, built-in auto-waiting, faster execution, easy handling of multiple tabs, powerful locators, API testing, and network interception, making it a modern and stable automation tool.

What are fixtures in playwright ?
A fixture is a **predefined setup** that Playwright provides to your test automatically.
Browser → creates environment
Context → isolates session
Page → performs actions

What is async ?
`async` is used before a function to indicate that the function contains asynchronous operations.

What is await ?
`await` is used before asynchronous actions to make JavaScript wait until that action is completed.

What is async and await ?
`async` is used to declare an asynchronous function, and `await` is used to pause execution until a promise is resolved. In Playwright, they ensure browser actions complete before moving to the next step.

What are locators ?
Locators are used to identify and interact with web elements on a webpage. In automation testing, before performing any action like click, type, or verify, we must first locate the element. That identification method is called a locator.

What are CSS selectors ?

CSS Selectors are patterns used to locate and identify HTML elements on a webpage based on their attributes, structure, or position.
(or)
CSS selectors are used to locate web elements using id, class, tag name, or attributes in automation testing.

For ID we use #
Example
```
await page.locator('#username').fill('admin');
```

For Class we use .
Example
```
await page.locator('.login-btn').click();
```

For Tag Name we use tagName directly
Example
```
await page.locator('input').first().fill('text');
```

For attribute we use tagName[key:'value']
Example
```
await page.locator('[type="password"]').fill('1234');
```

Modern Locators in Playwright (special locators)

Special locators in Playwright are built-in methods used to identify web elements based on user-visible properties like text, role, label, etc., instead of technical selectors like XPath or CSS.

1. getByRole()

- Finds elements based on **role like button , link**
- Most recommended locator

```
await page.getByRole('button', { name: 'Login' }).click();
```

2. getByText()

- Finds element using visible text

```
await page.getByText('Submit').click();
```

3. getByLabel()

- Used for input fields with labels

await page.getByLabel('Email').fill('test@mail.com');

4. getByPlaceholder()

- Finds input using placeholder text

await page.getByPlaceholder('Enter password').fill('123');

5. getByAltText()

- Used for images

await page.getByAltText('profile image').click();

6. getByTitle()

- Finds elements using title attribute

await page.getByTitle('Close').click();

Drop down in Playwright

A dropdown is a UI element that allows users to select one or more options from a list.

1. Static Dropdown (Select Tag)

- Uses `<select>` and `<option>` HTML tags
- Easiest to handle

await page.locator('#country').selectOption('India');

await page.locator('#country').selectOption([‘india’]);

Assertions in playwright

An **assertion** is used to **verify whether the expected result matches the actual result** in a test.

👉 If the condition is true → Test PASSES
👉 If the condition is false → Test FAILS

1. Page Assertions

Used to validate page-level properties.

```
await expect(page).toHaveTitle('Page Title');

await expect(page).toHaveURL('https://example.com');
```

2. Locator (Element) Assertions

Used to validate UI elements.

```
const element = page.locator('#id');


await expect(element).toBeVisible();      // Element is visible

await expect(element).toBeHidden();       // Element is hidden

await expect(element).toBeEnabled();      // Element is enabled

await expect(element).toBeDisabled();     // Element is disabled

await expect(element).toHaveText('Text'); // Exact text

await expect(element).toContainText('Te');// Partial text

await expect(element).toHaveAttribute('type', 'button');
```

3. Input Field Assertions

```
await expect(page.locator('#username')).toHaveValue('Bharat');
```

4. Checkbox / Radio Button Assertions

```
await expect(page.locator('#remember')).toBeChecked();
```

5. Count Assertions

Used to verify number of elements.

```
await expect(page.locator('.items')).toHaveCount(5);
```

Waits in Playwright

A **wait** is used to **pause test execution until a certain condition is satisfied**.

👉 Helps handle:

- Dynamic elements

- Page loading delays
- API/network delays

Why Wait is Needed?

- Web applications are **asynchronous**
- Elements may take time to load
- Without waits → tests may fail ❌

1. Auto Wait

Playwright **automatically waits** before performing actions.

✔ It checks:

- Element is visible
- Element is stable (not moving)
- Element is enabled
- Element is ready for action

Example

```
await page.click('#loginBtn');
```

Explicit Waits

Used when auto-wait is not enough.

---

waitForSelector()

Waits for element to appear in DOM.

```
await page.waitForSelector('#loginBtn');
```

---

waitForLoadState()

Waits for page loading states.

```
await page.waitForLoadState('load');              // full page load

await page.waitForLoadState('domcontentloaded');  // DOM ready
```

```
await page.waitForLoadState('networkidle');       // no network calls
```

---

waitForURL()

Waits for URL to change.

```
await page.waitForURL('**/dashboard');
```

---

waitForTimeout() ❌ (Hard Wait)

Fixed delay (not recommended)

```
await page.waitForTimeout(3000);
```

Drag and Drop in Playwright

Drag and Drop is a mouse action where an element is:
👉 Clicked and held (dragged)
👉 Moved to another location
👉 Released (dropped)

Why is it Used?

- Moving items between lists
- Uploading files (drag UI)

1. Using `dragAndDrop()`

Syntax

```
await page.dragAndDrop(source, target);
```

Example

```
await page.dragAndDrop('#source', '#target');
```

Automatically performs:

- Mouse down
- Move
- Mouse up

2. Using Locator

const source = page.locator('#source');

const target = page.locator('#target');

await source.dragTo(target);

👉 More readable and recommended

3. Using Mouse (Low-Level)

Used when `dragAndDrop()` doesn't work.

skip, only, fixme in Playwright

These are used to **control test execution**.

1. `test.skip()`

Definition:

Used to **skip a test** (test will not run).

Syntax:

test.skip('test name', async ({ page }) => {

// test code

});

2.test.only()

Definition:

Runs **only this test** and skips all others.

Syntax:

```
test.only('test name', async ({ page }) => {
 // test code
});
```

3. test.fixme()

Definition:

Marks a test as **expected to fail** and skips execution.

Syntax:

```
test.fixme('test name', async ({ page }) => {
 // test code
});
```

Screenshot in Playwright

Definition

A **screenshot** is used to **capture the current state of the webpage or element** during test execution.

Useful for:

- Debugging
- Reporting
- Evidence of test execution

Types of Screenshots

1. Full Page Screenshot
2. Element Screenshot
3. Visible Page screenshot

Syntax

Full Page (Complete Scroll)

await page.screenshot({ path: 'fullpage.png', fullPage: true });

2. Element Screenshot

Syntax:

await locator(“locator_name”).screenshot({ path: 'element.png' });

3.Visible Page Screenshot

Syntax:

await page.screenshot({ path: 'filename.png' });

What is an Alert?

An **alert** is a popup message displayed by the browser.

Types:

- Simple Alert
- Confirmation Alert
- Prompt Alert

Important Point

Playwright **automatically handles alerts** if no listener is added.

Default behavior:

- Automatically **accepts alert**

Handling Alerts (Using Event Listener)

Syntax

page.on('dialog', async dialog => {

```
 // action
});
```

1. Accept Alert

```
page.on('dialog', async dialog => {
 await dialog.accept();
});
```

2. Dismiss Alert

```
page.on('dialog', async dialog => {
 await dialog.dismiss();
});
```

3. Get Alert Text

```
page.on('dialog', async dialog => {
 console.log(dialog.message());
 await dialog.accept();
});
```

4. Handle Prompt Alert (Send Input)

```
page.on('dialog', async dialog => {
 await dialog.accept('Hello');
});
```

Types of Alerts

1. Simple Alert

- Only OK button

```
await dialog.accept();
```

2. Confirmation Alert

- OK and Cancel buttons

```
await dialog.accept();   // OK
await dialog.dismiss();  // Cancel
```

3. Prompt Alert

- Input field + OK/Cancel

```
await dialog.accept('Text Input');
```

Page Object Model (POM)

POM (Page Object Model) is a design pattern used to **organize automation code**.

Each web page is represented as a **separate class (file)**
Contains:

- Locators

- Methods (actions)

Why Use POM?

Improves code readability
Reusable code
Easy maintenance
Reduces duplication

Mouse Actions in Playwright

Definition

Mouse actions are used to **simulate user interactions using a mouse** on web elements.

Examples:

- Click
- Double Click
- Right Click
- Hover
- Drag and Drop

1. Click

```
await page.click('#btn');
```

Clicks on an element

2. Double Click

```
await page.dblclick('#item');
```

Performs double click

3. Right Click (Context Click)

```
await page.click('#element', { button: 'right' });
```

Opens context menu

4. Hover

await page.hover('#menu');

Moves mouse over an element

5. Drag and Drop

await page.dragAndDrop('#source', '#target');

Drags element from source to target

Keyboard Actions in Playwright

What are Keyboard Actions?

Keyboard actions are used to **simulate user key presses** like:

- Typing text
- Pressing Enter
- Using shortcuts (Ctrl + C, Ctrl + V)

1. Typing Text

`fill()` method

await page.fill('#username', 'bharat');

- Clears existing text
- Types new value
- Fast execution

`type()` method

await page.type('#username', 'bharat');

- Types character by character
- Mimics real user typing

`PressSequentially()` method

await page.locator().pressSequentially(‘value’) ;

- When you have to type letter by letter

Difference

Method	Behavior
fill()	Clears + types instantly
type()	Types slowly like user

2. Pressing Keys

await page.keyboard.press('Enter');

Common Keys:

- `Enter`
- `Tab`
- `Escape`
- `ArrowUp`
- `ArrowDown`
- `Backspace`

3. Keyboard Shortcuts

await page.keyboard.press('Control+A'); // Select all

await page.keyboard.press('Control+C'); // Copy

await page.keyboard.press('Control+V'); // Paste

For Mac:

await page.keyboard.press('Meta+A');

4. Key Down & Key Up

Used for advanced control:

await page.keyboard.down('Shift');

await page.keyboard.press('KeyA');

await page.keyboard.up('Shift');

Output: A

5. Typing with Delay

await page.type('#search', 'Playwright', { delay: 100 });

- Types slowly
- Useful for debugging and real-time simulation

6. Real-Time Example (Login)

await page.fill('#username', 'bharat');

await page.keyboard.press('Tab');

await page.fill('#password', '12345');

await page.keyboard.press('Enter');

What is Data-Driven Testing?

Data-Driven Testing (DDT) is a technique where:
The same test script is executed **multiple times with different input data**

Why use Data-Driven Testing?

- Avoid writing duplicate test cases
- Increase test coverage
- Easy to test multiple scenarios
- Data is separated from test logic

1. Using Array (Basic Method) / 2d array

2. Using JSON File

3. Using CSV (Advanced)

4. Using XLSX

Benefits?

- Reusability
- Better coverage
- Cleaner code

What is Playwright Architecture?

Playwright architecture defines **how Playwright interacts with browsers** to execute automation scripts.

It follows a **Client–Server Architecture**

High-Level Flow

Test Script → Playwright Client → Browser Server → Browser→ test execution on website

Main Components

Java Script (User Code)

- Written by tester (JavaScript / TypeScript / Python / Java)
- Contains test logic

```
test('Login Test', async ({ page }) => {

 await page.goto('https://example.com');

});
```

Playwright Client

- Acts as a **bridge**
- Converts your code into commands
- Sends commands to browser

Example:

```
await page.click('#login');
```

➡ Converted into low-level instruction

Browser Server

- Playwright launches a browser instance
- Controls browser via **WebSocket connection**
- Sends and receives responses

Browser (Chromium / Firefox / WebKit)

- Actual browser where actions happen
- Executes commands like:
    - Click
    - Type
    - Navigate

Communication Flow

1. Test script sends command
2. Client converts it
3. Sent via WebSocket
4. Browser executes
5. Response comes back

Supported Browsers

- Chromium
- Firefox
- WebKit

One script → multiple browsers

Key Features of Architecture

1. Auto-Waiting

- Playwright waits automatically for elements

```
await page.click('#login');
```

No need for manual waits

2. Headless & Headed Mode

- Headless → runs in background
- Headed → visible browser

3. Parallel Execution

- Run multiple tests at same time
- Faster execution

4. Isolation (Browser Context)

- Each test runs in separate environment
- No data conflict

Browser Context (Very Important)

- Like **incognito mode**
- Each test gets fresh session

```
const context = await browser.newContext();

const page = await context.newPage();
```

Feature	Playwright	Selenium	Cypress
Speed	Very fast (direct browser control)	Slower (WebDriver layer)	Fast
Auto-Waiting	Built-in (no need for waits)	Manual waits needed	Partial
Browser Support	Chromium, Firefox, WebKit	All major browsers	No Firefox/WebKit (limited)
Parallel Execution	Built-in	Needs setup (Grid)	Limited
Architecture	Direct communication (WebSocket)	WebDriver protocol	Runs inside browser
Handling Multiple Tabs	Easy	Complex	Limited

Mobile Testing	Supported	Limited	Not supported
Network Interception	Strong support	Limited	Good
API Testing	Built-in	Not available	Limited
Test Isolation	Browser Context (incognito-like)	Not by default	Limited
Cross-Origin Testing	Supported	Complex	Restricted
Headless Mode	Yes	Yes	Yes

Running Tests in Multiple Environments (Playwright)

What is Multiple Environment Testing?

Running the **same test** against different environments like:

- QA (Testing)
- Staging (Pre-production)
- Prod (Live)

Why do we need this?

- Test same feature in different setups
- Catch environment-specific bugs
- Avoid code duplication
-

How do you run tests in multiple environments?

Using:

- `.env` files
- Playwright config projects
- Command line parameters